Guía de examen UT03 · Hoja 9 (DWES)

Oct 22, 2026 · @Roberto

Todos los ejercicios de la hoja 9 son el mismo molde: un formulario autogestionado en un único archivo PHP que pasa los datos por cuatro etapas (limpieza, validación, errores y salida). Si dominas las piezas de esta guía y el molde, puedes resolver cualquier variante del examen.

Qué te van a pedir y cómo usar esta guía

Los cuatro ejercicios de la hoja 9 (`UT03-Ejercicios_9.pdf`) comparten siempre los mismos ingredientes. Si el examen es parecido, te pedirán una combinación de estos:

Ingrediente	Ejercicio de la hoja 9 donde sale	Qué tienes que saber hacer
Una sola página PHP con el formulario y el proceso	1, 2, 3 y 4	Formulario autogestionado con `$_SERVER['REQUEST_METHOD']` y `action` con `$_SERVER['PHP_SELF']`
Arrays con los datos del problema	1 (monedas, cambios), 2 y 3 (películas, imágenes)	Arrays asociativos y anidados, recorrerlos con `foreach`
Array de errores y array de resultados	1 y 4	Etapa 3: `$errores['campo'] = 'mensaje'` y `$datos['campo'] = valor`
Validar números (mayor que 0, coma o punto)	1 y 4	`trim` , `str_replace` , `filter_var` y comparar con `=== false`
Elegir de una lista (`select`)	1	Lista blanca: el valor tiene que existir en tu array
Inputs creados con un bucle y enviados como array	4	`name="numeros[]"` + `foreach` / `for` en el HTML
Buscar por texto	2 y 3	`mb_stripos` dentro de un `foreach`
Salida segura y formateada	Todos	`htmlspecialchars` (función `h()`) y `number_format`

Orden de estudio recomendado:

1. Lee los **Fundamentos** (secciones 1 a 7). Cada pieza tiene un ejemplo mínimo y la página del apunte donde aparece.
2. Memoriza el **molde de las cuatro etapas**: es el esqueleto que copias al empezar cualquier ejercicio.
3. Repasa los **cuatro ejercicios paso a paso** en el mismo orden en que los escribirías en el examen.
4. Pasa la **checklist** y haz el **ejercicio tipo examen** sin mirar las soluciones. Después pruébalo con los casos de la tabla.

Las soluciones completas y probadas están en los archivos `ejercicio1.php` a `ejercicio4.php` que te pasé. Lo que aquí se marca como **[no está en el material]** es un aviso mío que no sale en tus apuntes; lo marcado **[lo exige el enunciado, no aparece en los apuntes]** lo pide la hoja 9 aunque no esté explicado.

Fundamentos 1 · PHP dentro de HTML, variables y tipos

Un ejercicio de examen es un archivo `.php` con dos partes: arriba el PHP que prepara los datos y abajo el HTML que los muestra. PHP solo se ejecuta dentro de sus etiquetas.

Etiquetas

Escribes	Qué hace	Dónde sale
`<?php ... ?>`	Abre y cierra un bloque de código PHP	Chuleta de PHP, pág. 2
`<?= $x ?>`	Etiqueta corta: equivale a `<?php echo $x; ?>`. Se usa dentro del HTML	Chuleta de PHP, pág. 2
`echo` / `print`	Imprimen texto	UT02_Teoría, pág. 6
`// comentario` y `/* comentario */`	Comentarios de una línea y de varias	Chuleta de PHP, págs. 2-3

```
<?php
declare(strict_types=1);   // SIEMPRE la primera línea (ver más abajo)

$titulo = 'Conversor';     // parte PHP: preparar datos
?>
<!DOCTYPE html>
<html lang="es">
<body>
    <h1><?= $titulo ?></h1>  <!-- parte HTML: mostrar datos -->
</body>
</html>
```

Variables y tipos

- Una variable empieza por `$`. El tipo no se declara: PHP lo deduce y puede cambiar (UT02_Teoría, pág. 5).
- Convención del curso: **lowerCamelCase** (`$tiposCambio`, `$totalCajas`) (UT02_Teoría, pág. 5).
- Tipos que usarás: `int` (7), `float` (12.5), `string` ('hola'), `bool` (true/false), `null` (sin valor) y `array`.
- **Casting** (convertir de tipo) poniendo el tipo entre paréntesis: `(string) 1996` da `'1996'`; `(float) '3.5'` da `3.5` (UT02_Teoría, pág. 5).
- Concatenar texto con el **punto**: `'Hola ' . $nombre`. Y `.=` añade al final (UT02_Teoría, pág. 6).

Operadores que vas a necesitar

Operador	Para qué	Ejemplo
`+ - * / %`	Aritmética (`%` = resto)	`$a / $b * $c`
`+=`	Sumar y asignar	`$suma += $numero;`
`< > <= >=`	Comparar	`$cantidad <= 0`
`===` y `!==`	Idéntico / no idéntico: compara valor **y tipo**	`$valor === false`
`&&`	Y lógico: se cumplen las dos	`$valor !== false && $valor % 2 === 0`
`||`	O lógico: basta con una	`$nota < 0 || $nota > 10`

Operador	Para qué	Ejemplo
`!`	Negación	`!empty($errores)`

Fuentes: UT02_Teoría, pág. 6; Chuleta de PHP, págs. 8-9.

`declare(strict_types=1);`

Activa el **modo estricto de tipos**: si una función pide `int` y le pasas un `float`, PHP lanza un error en vez de convertirlo (UT03-Programación código embebido.pdf, pág. 12). Todos los ejercicios resueltos lo ponen en la primera línea.

Consecuencia práctica: `h()` recibe `string`. Para imprimir un número con `h()` hay que convertirlo antes: `h((string) $info['anio'])` (zip del buscador, `vistas/resultados.php`).

Fundamentos 2 · Arrays

En la hoja 9 **todo son arrays**: los datos del problema, los errores, los resultados e incluso lo que envía el formulario (`$_POST` y `$_GET` son arrays). Un array guarda varios valores, cada uno con su **clave**.

Los tres tipos que usarás

```
<?php
// 1) INDEXADO: las claves son 0, 1, 2... (PHP las pone solo)
$nombres = ['Lucía', 'Victor', 'Efren'];
echo $nombres[0] . '<br>';              // Lucía

// 2) ASOCIATIVO: tú eliges la clave (normalmente un texto)
$monedas = [
    'EUR' => 'Euro',
    'USD' => 'Dólar (USD)',
];
echo $monedas['USD'] . '<br>';          // Dólar (USD)

// 3) ANIDADO (multidimensional): cada valor es otro array
$peliculas = [
    'Volver' => ['anio' => 2006, 'sinopsis' => 'Drama de Pedro
Almodóvar...'],
];
echo $peliculas['Volver']['anio'] . '<br>';   // 2006  (una clave por cada
nivel)
```

En el navegador debe salir una línea por `echo`: Lucía, Dólar (USD) y 2006. El `. '<br>'` concatena un salto de línea (UT02_Teoría, pág. 6). Ojo: el nombre de la variable y las claves tienen que escribirse **exactamente igual** al crearlas y al leerlas. `$nombre` no es `$nombres`, y `'año'` no es `'anio'`.

Fuentes: UT03-Programación código embebido.pdf, págs. 17-19 (arrays y arrays de varias dimensiones, ejemplo `$ciclos["DAW"]["DWES"]`).

Crear, añadir y leer

Quiero…	Código	Fuente
Array vacío	`$errores = [];`	Todos los ejercicios resueltos
Añadir con clave	`$errores['cantidad'] = 'Mensaje';`	UT03, pág. 19
Añadir al final (clave automática)	`$lista[] = 'valor';`	UT03, pág. 19
Leer un valor	`$monedas['EUR']`	UT03, pág. 18
Ver todo el array (para depurar)	`print_r($array);` o `var_dump($array);`	UT03, págs. 17 y 20

Funciones de arrays que necesitas

Función	Qué hace	Ejemplo y resultado	Fuente
`count($a)`	Número de elementos	`count(range(10, 20))` da 11	UT03, pág. 21
`range($ini, $fin)`	Crea un array con un intervalo	`range(1, 10)` da `[1, 2, ..., 10]`	UT03, págs. 20-21
`isset($a[$clave])`	`true` si la clave existe y no es `null`	`isset($monedas['GBP'])`	UT03, pág. 21
`in_array($valor, $a)`	`true` si el VALOR está en el array	`in_array('suma', $operaciones)`	UT03, pág. 21
`empty($a)`	`true` si el array está vacío	`if (empty($errores))`	Chuleta de PHP, pág. 5
`is_array($x)`	`true` si `$x` es un array	`is_array($_POST['numeros'] ?? null)`	UT03, pág. 52; zip `EjemplosFilter_var`

Función	Qué hace	Ejemplo y resultado	Fuente
`unset($a[$clave])`	Borra un elemento	`unset($datos['numeros'])`	UT03, pág. 21

`isset` o `in_array`: cuál uso. `isset($monedas[$codigo])` mira si existe una **clave**; `in_array($op, $lista)` mira si existe un **valor**. Para un `select` cuyos códigos son las claves de tu array (ejercicio 1) usas `isset`. Si tienes una lista simple de valores permitidos (`['suma','resta']`, ejercicio 8 de la hoja 8) usas `in_array`.

Truco del examen: guarda los errores y los datos con la misma clave que el campo

```
$errores = [];   // un mensaje por cada campo que falla
$datos   = [];   // un valor por cada campo correcto

$errores['cantidad'] = 'Introduce una cantidad numérica válida.';   // clave
= name del input
$datos['origen']     = 'EUR';
```

Así, en el HTML, `$errores['cantidad']` va justo al lado del input `name="cantidad"` (02-RA3-ValidacionFormularios.pptx, diap. 8).

Fundamentos 3 · Condicionales y bucles

Con condicionales decides **qué camino** sigue el programa (¿hay errores?, ¿es GET o POST?). Con bucles **recorres arrays** y **creas HTML repetido** (opciones de un `select`, cajas de un formulario, resultados de una búsqueda).

if / elseif / else

```
<?php
declare(strict_types=1);

// En el ejercicio 1, $cantidad sale de validarNumero() (Fundamentos 7): un
float o null.
// Para probar este bloque suelto, dale tú un valor y ve cambiándolo: null,
-5.0 o 50.0
$cantidad = 50.0;

$errores = [];
$datos = [];

if ($cantidad === null) {
    $errores['cantidad'] = 'Introduce una cantidad numérica válida.';
```

```
} elseif ($cantidad <= 0) {
    $errores['cantidad'] = 'La cantidad debe ser mayor que cero.';
} else {
    $datos['cantidad'] = $cantidad;
}

print_r($errores);   // qué ha fallado
echo '<br>';
print_r($datos);     // qué es correcto
```

Resultado con cada valor: `null` → error «Introduce una cantidad numérica válida.»; `-5.0` → error «La cantidad debe ser mayor que cero.»; `50.0` → `$errores` vacío y `$datos['cantidad']` = 50. `print_r` muestra el array entero (UT03, pág. 17).

El orden importa: primero lo más general (no es un número) y después lo concreto (es un número, pero ⩽ 0). Así cada fallo tiene **su propio mensaje**. Fuente: UT03-Programación código embebido.pdf, pág. 2.

Ternario y fusión de null

Operador	Se lee	Ejemplo	Fuente
`condición ? A : B`	«Si se cumple, A; si no, B»	`$valor === false ? null : $valor`	UT03, pág. 3
`$x ?? 'defecto'`	«Si `$x` existe y no es null, `$x`; si no, el valor por defecto»	`$_POST['cantidad'] ?? ''`	UT03, pág. 59

El `??` es el que **evita el aviso «Undefined array key»** cuando un campo no llega en el formulario. Úsalo siempre al leer `$_POST` o `$_GET`.

foreach: recorrer un array

```
// Solo los valores
foreach ($errores as $mensaje) {
    echo $mensaje;
}

// Clave y valor a la vez (lo que más usarás)
foreach ($monedas as $codigo => $descripcion) {
    echo $codigo . ' - ' . $descripcion;   // EUR - Euro, USD - Dólar
(USD)...
}
```

Fuente: UT03, pág. 6 y pág. 19 («recorriendo los elementos y sus valores clave de forma simultánea»).

for: repetir un número de veces conocido

```
for ($i = 0; $i < 10; $i++) {   // inicio; condición; incremento
    $entrada[$i] = trim($numeros[$i] ?? '');
}
```

Las tres partes: se ejecuta una vez al empezar, se evalúa antes de cada vuelta y se ejecuta al final de cada vuelta (UT03, pág. 6). También existen `while` y `do...while` (UT03, pág. 5), pero en la hoja 9 no hacen falta.

Acumular con un bucle (sumar, contar)

```
$suma = 0;                    // 1) inicializa FUERA del bucle
foreach ($numeros as $numero) {
    $suma += $numero;         // 2) acumula dentro
}
// 3) usa el resultado después
```

Es el mismo patrón que `concatenar()` de UT03, pág. 9 (`$resultado .= $palabra`), cambiando `.=` por `+=` . Para **contar** (por ejemplo, aprobados) haces lo mismo con `$aprobadas++` dentro de un `if` .

Sintaxis alternativa dentro del HTML

Cuando el bucle o el `if` envuelven HTML, en vez de llaves se usan dos puntos y `endforeach;` / `endif;` . Se lee mucho mejor:

```
<?php if (!empty($errores)): ?>
    <ul>
        <?php foreach ($errores as $mensaje): ?>
            <li><?= h($mensaje) ?></li>
        <?php endforeach; ?>
    </ul>
<?php endif; ?>
```

Fuente: zip `06-EjercicioBuscadorPeliculas…` (`Ejercicio2Original.php`) y zip `05-FormularioAutogestionadoConValidaciones` (`formularioValidacion.php`). Cada `if (...):` necesita su `endif;` y cada `foreach (...):` su `endforeach;` : si falta uno, la página entera da error de sintaxis.

Fundamentos 4 · Funciones

Una función agrupa una tarea con un nombre para reutilizarla. En los ejercicios resueltos cada etapa tiene sus funciones: `validarNumero()`, `gestionarErrores()`, `h()`...

Anatomía de una función con tipos

```
//      nombre        parámetro con tipo     tipo que devuelve
function validarEdad(string $edad): ?int
{
    $v = filter_var($edad, FILTER_VALIDATE_INT,
            ['options' => ['min_range' => 18, 'max_range' => 65]]);

    return $v === false ? null : $v;   // return termina la función y
devuelve el valor
}

$edad = validarEdad('30');   // llamada: $edad vale 30
```

Pieza	Significado	Fuente
`function nombre(...)`	Define la función	UT03, pág. 7
`string $edad`	Parámetro: el valor que recibe, con su tipo	UT03, págs. 8 y 11
`: ?int`	Tipo de lo que devuelve. El `?` delante significa **«o null»** (tipo anulable)	UT03, pág. 12; Chuleta de PHP, pág. 4
`return`	Devuelve el valor y termina la función	UT03, pág. 8
`$iva = 0.21` en un parámetro	Valor por defecto si no lo pasas	UT03, pág. 9

La convención de tus apuntes: válido = el valor, inválido = null

Todas las funciones `validarXxx()` de los zips devuelven **el valor ya convertido** si es correcto y **`null`** si no lo es (`Validaciones.php` del zip `05-FormularioAutogestionadoConValidaciones` y `validaciones.php` del zip `06-Ejercicio8hoja8`). Por eso después se pregunta siempre:

```
$nombre = validarNombre($entrada['nombre']);
if ($nombre === null) {
    $errores['nombre'] = 'El nombre solo puede contener letras y espacios.';
} else {
    $datos['nombre'] = $nombre;
}
```

Usa `=== null` y no `if (!$nombre)`: un `0` es válido y `!0` daría `true` (ver Fundamentos 7).

Por qué los arrays se pasan como parámetro

Las variables de fuera **no se ven dentro** de una función: el ámbito es local (UT02_Teoría, pág. 8; UT03, pág. 7). Existe `global`, pero tus apuntes recomiendan pasar parámetros: «Siempre es mejor utilizar argumentos o parámetros al hacer la llamada» (UT03, pág. 8).

```
// MAL: $monedas no existe dentro de la función
function validarMoneda(string $codigo): ?string
{
    return isset($monedas[$codigo]) ? $codigo : null;   // $monedas es
desconocida aquí
}

// BIEN: se pasa como parámetro
function validarMoneda(string $codigo, array $monedas): ?string
{
    return isset($monedas[$codigo]) ? $codigo : null;
}
```

Excepción: `$_POST`, `$_GET` y `$_SERVER` son **superglobales** y se ven en todas partes (UT03, pág. 22). Por eso `limpiarBusqueda()` puede leer `$_GET['q']` sin recibirlo.

Devolver dos cosas a la vez

Una función devuelve un solo valor, así que para devolver errores **y** datos se devuelve un array con las dos cosas (zip `06-Ejercicio8hoja8`, `funciones.php`):

```
function gestionarErrores(array $entrada): array
{
    $errores = [];
    $datos = [];
    // ... validar y rellenar ...
    return ['errores' => $errores, 'datos' => $datos];
}

$validacion = gestionarErrores($entrada);
$errores = $validacion['errores'];
$datos   = $validacion['datos'];
```

Fundamentos 5 · Cadenas (strings)

Todo lo que llega de un formulario es **texto**, aunque el usuario escriba un número. Estas funciones preparan ese texto y buscan dentro de él.

Función	Qué hace	Ejemplo → resultado	Fuente
`trim($s)`	Quita espacios al principio y al final	`trim(' 12,5 ')` → `'12,5'`	02-RA3-ValidacionFormularios.pptx, diap. 4; Chuleta de PHP, pág. 10
`str_replace($buscar, $poner, $s)`	Reemplaza un texto por otro	`str_replace(',', '.', '12,5')` → `'12.5'`	UT03, págs. 15-16
`mb_strlen($s)`	Número de **caracteres** (bien con tildes y ñ)	`mb_strlen('áeiou')` → 5	UT03, pág. 17
`mb_stripos($texto, $buscar)`	Posición de la primera coincidencia **sin distinguir mayúsculas**; `false` si no está	`mb_stripos('Volver', 'VOL')` → 0	zip del buscador (`buscador.php`)

Por qué las versiones `mb_`

`strlen` cuenta **bytes**, no letras: `strlen('áeiou')` da 6 porque la á ocupa dos bytes en UTF-8 (UT03, pág. 17). Con títulos como «El espíritu de la colmena», usa siempre las versiones `mb_`: `mb_strlen` y `mb_stripos`.

La trampa de `mb_stripos`: la posición 0

[No está explicado en el material; el código de tus zips ya lo hace bien.] Si la coincidencia está al principio, `mb_stripos` devuelve `0`, que PHP considera «falso». Por eso se compara con `!== false`:

```
if (mb_stripos($titulo, $q) !== false) {   // BIEN: 0 cuenta como encontrado
    $encontradas[$titulo] = $info;
}

if (mb_stripos($titulo, $q)) {             // MAL: 'Volver' buscando 'vol' NO
se encontraría
```

Fundamentos 6 · Formularios

Cómo viajan los datos

- El atributo **`name`** de cada campo es la **clave** con la que llega a PHP: `<input name="cantidad">` → `$_POST['cantidad']` (pptx, diap. 3; UT03, págs. 57-58).
- **`method="get"`**: los datos van en la URL (`ejercicio2.php?q=volver`) y se leen en `$_GET`. Conviene para **consultar** (búsquedas).
- **`method="post"`**: van en el cuerpo de la petición y se leen en `$_POST`. Conviene para **enviar** datos (conversor, sumas).
- Fuente: «Es conveniente usar GET para la recuperación y POST para el envío de información» (UT03, pág. 57).
- **Todo llega como texto (string)**, incluso los números. Por eso hay que validarlo y convertirlo.

Formulario autogestionado (una sola página)

El mismo archivo muestra el formulario **y** lo procesa. Distingue las dos situaciones con `$_SERVER['REQUEST_METHOD']` (UT03, pág. 22; pptx, diap. 9):

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // el usuario ha pulsado el botón: limpiar, validar, calcular
}
// si es GET (primera visita) no entra: se muestra el formulario con valores
iniciales
```

El `action` apunta a la propia página con `$_SERVER['PHP_SELF']` **escapado** (lo exige el enunciado del ejercicio 1; así está en `Ejercicio2Original.php`):

```
<form method="post" action="<?= h($_SERVER['PHP_SELF']) ?>">
```

Devolver lo escrito (sticky value)

Tras un error, el usuario no debe perder lo que escribió. El `value` se rellena con **lo que escribió** (ya limpio), escapado:

```
<input type="text" name="cantidad" value="<?= h($entrada['cantidad']) ?>">
```

Se saca de `$entrada` (o de `$_POST`), **nunca de `$datos`**: `$datos` solo tiene los campos correctos y un campo erróneo aparecería vacío (pptx, diap. 10).

Select generado desde un array, con la opción elegida marcada

```
<select name="origen">
    <?php foreach ($monedas as $codigo => $descripcion): ?>
        <option value="<?= h($codigo) ?>" <?= $entrada['origen'] === $codigo
? 'selected' : '' ?>>
            <?= h($codigo . ' - ' . $descripcion) ?>
        </option>
    <?php endforeach; ?>
</select>
```

- `value` = lo que se envía (el código); el texto entre etiquetas = lo que ve el usuario.
- `selected` marca la opción elegida. Es el mismo truco que `checked` en los radios del ejercicio 8 de la hoja 8 (zip `06-Ejercicio8hoja8`, `funciones.php`).
- `select` y `option`: UT03, pág. 54.

Varios campos con el mismo nombre: `name="numeros[]"`

Si el nombre acaba en `[]`, PHP agrupa todos esos campos en **un array**:

```
<input name="numeros[]" value="1">
<input name="numeros[]" value="2">
<input name="numeros[]" value="3">
```

```
$_POST['numeros'];   // ['1', '2', '3']  (posiciones 0, 1, 2 en el orden del
formulario)
```

Fuente: zip `05-EjemplosFilter_var` (`formularioFilterVar.html`, checkbox `numeros[]`, y su validación en `validacionesFilterVar.php`). Antes de recorrerlo, comprueba que de verdad es un array con `is_array()`: alguien puede enviar `numeros=texto`.

Fundamentos 7 · Salida segura con h() y validación con filter_var

Dos reglas que valen puntos en todos los ejercicios: **se valida al entrar** (`filter_var`) y **se escapa al salir** (`htmlspecialchars`). No son lo mismo: validar decide si el dato sirve, escapar impide que el navegador lo ejecute.

htmlspecialchars y la función h()

Si imprimes tal cual lo que escribe el usuario y escribe `<script>alert('hackeado')</script>` , el navegador **ejecuta** ese código (ataque XSS). `htmlspecialchars` convierte los caracteres especiales en entidades y el navegador los muestra como texto (UT03, págs. 60-61):

Carácter	Se convierte en
`&`	`&`
`<`	`<`
`>`	`>`
`"`	`"`
`'`	`'` (solo con `ENT_QUOTES`)

Como se usa en cada impresión, se envuelve en una función corta. Es la de tus apuntes con tipo `string` añadido, como en los zips:

```
function h(string $s): string
{
    return htmlspecialchars($s, ENT_QUOTES, 'UTF-8');
}
```

- `ENT_QUOTES` : escapa también la comilla simple. Sin él, un valor dentro de `value='...'` podría cerrar el atributo e inyectar código (UT03, pág. 61).
- `'UTF-8'` : para que tildes y ñ no se estropeen (UT03, pág. 61).
- **Qué se escapa:** todo lo que imprimes y viene de fuera o de tus arrays: valores de los `input` , mensajes de error, resultados, opciones del `select` , rutas de imágenes y `$_SERVER['PHP_SELF']` .
- **Qué no:** los números que ya pasan por `number_format` .

filter_var: validar y convertir de una vez

`filter_var($dato, FILTRO, opciones)` devuelve **el valor ya convertido** si es válido y **`false`** si no lo es. No modifica ni limpia el dato (pptx, diaps. 5-6).

Necesito validar	Filtro	Opciones	Fuente
Número decimal	`FILTER_VALIDATE_FLOAT`	ninguna	pptx, diap. 5; zip `EjemplosFilter_var`
Entero (con rango)	`FILTER_VALIDATE_INT`	`['options' => ['min_range' => 0, 'max_range' => 100]]`	pptx, diap. 5; zip `EjemplosFilter_var`
Texto con un patrón	`FILTER_VALIDATE_REGEXP`	`['options' => ['regexp' => '/^[a-záéíóúüñ]+$/i']]`	zip `FormularioAutogestionado` (`validarNombre`)
Email	`FILTER_VALIDATE_EMAIL`	ninguna	pptx, diaps. 5-6

Compara SIEMPRE con `=== false`

```
$valor = filter_var('0', FILTER_VALIDATE_INT);   // int(0): válido

if ($valor === false) { ... }   // BIEN: solo entra si de verdad no es válido
if (!$valor) { ... }            // MAL: el 0 también entraría como "inválido"
```

«Válido: devuelve el valor. No válido: devuelve false. Compara con === false» (pptx, diap. 5).

Números con coma o con punto (ejercicio 1)

`FILTER_VALIDATE_FLOAT` solo entiende el **punto**: `'12,5'` da `false`. [No está en el material, comprobado ejecutándolo.] La solución con lo que tienes en los apuntes es cambiar la coma por un punto antes de validar:

```
function validarNumero(string $texto): ?float
{
    $texto = str_replace(',', '.', $texto);            // '12,5' -> '12.5'
    $valor = filter_var($texto, FILTER_VALIDATE_FLOAT);  // 12.5 o false
    return $valor === false ? null : $valor;
}
```

[No está en el material, comprobado ejecutándolo] `FILTER_VALIDATE_FLOAT` acepta notación científica: `'1e3'` se valida como 1000. Para el examen no es un problema, pero no te sorprendas si lo ves.

Un rango con decimales

Para «mayor que cero» o «entre 0 y 10» con decimales, valida primero que es número y después compara a mano, con **dos mensajes distintos**:

```
$nota = validarNumero($texto);
if ($nota === null) {
    $errores['nota'] = 'Introduce una nota numérica.';
} elseif ($nota < 0 || $nota > 10) {
    $errores['nota'] = 'La nota debe estar entre 0 y 10.';
} else {
    $datos['nota'] = $nota;
}
```

Es el mismo esquema que el «entero par» del zip `EjemplosFilter_var` (filtro + comprobación manual) y que las validaciones transversales de la pptx, diap. 7 (rango numérico).

number_format para la salida

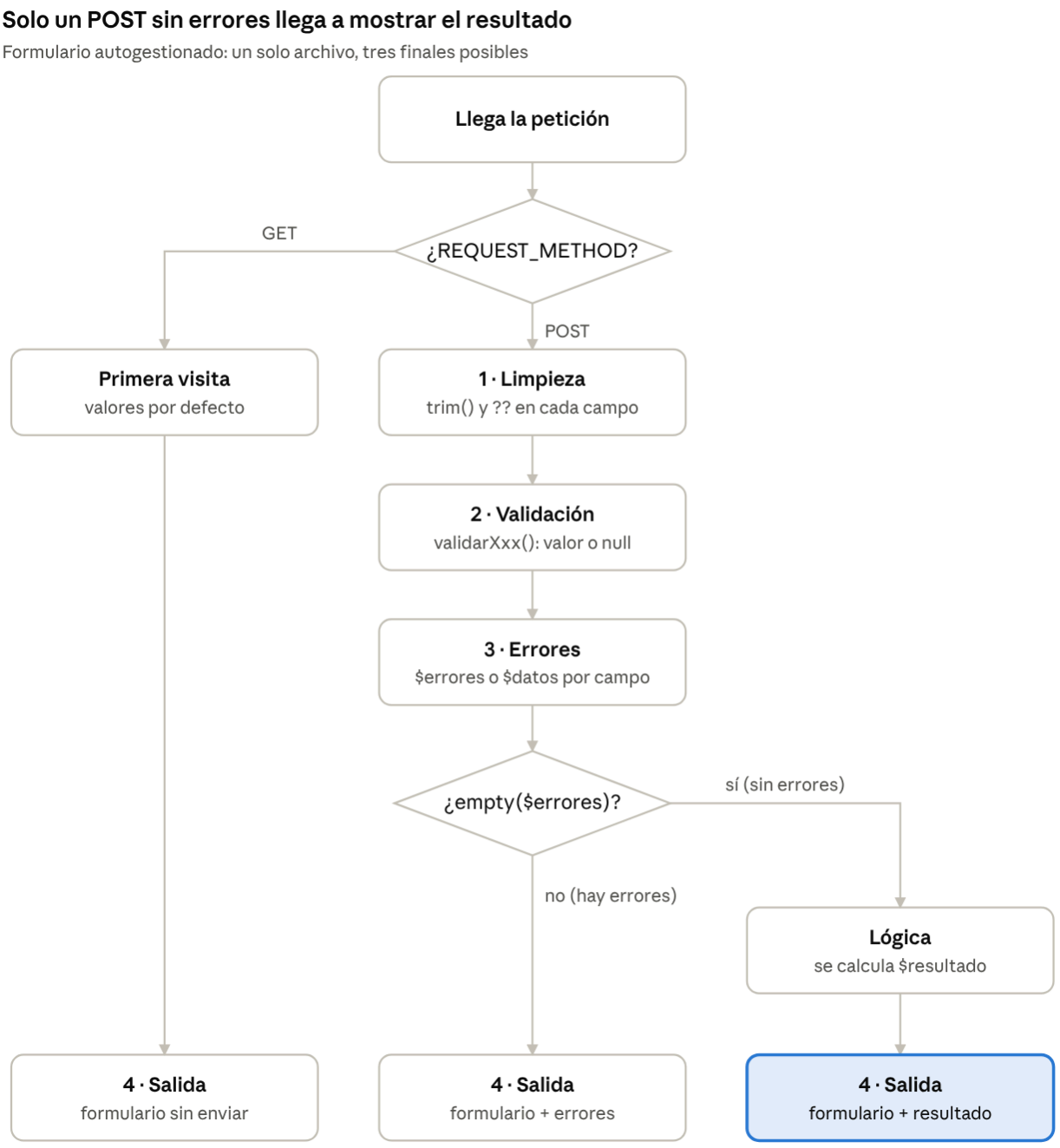
```
number_format(1234.5, 2, ',', '.');   // '1.234,50'
//            número, decimales, separador decimal, separador de miles
```

[lo exige el enunciado, no aparece en los apuntes] Solo sale en el enunciado del ejercicio 1. Devuelve un string con el formato español.

El molde: las cuatro etapas

Todo dato que llega de un formulario pasa por las mismas cuatro etapas y en el mismo orden (02-RA3-ValidacionFormularios.pptx, diaps. 2 y 10):

Etapa	Pregunta que responde	Qué escribes
1. Limpieza	¿Qué le sobra al dato?	`trim($_POST['campo'] ?? '')`
2. Validación	¿Cumple lo que pide el enunciado?	`validarXxx()` con `filter_var`, `isset`, rangos…
3. Errores	¿Qué ha fallado?	`$errores['campo'] = 'mensaje'` o `$datos['campo'] = valor`
4. Salida	¿Cómo lo muestro sin riesgo?	`h()`, valores escritos en `value`, errores junto al campo y, si no hay errores, el resultado



Flujo de un formulario autogestionado · 4 etapas, 2 decisiones

La primera visita (GET) solo muestra el formulario. Un envío (POST) pasa siempre por limpieza, validación y errores; el resultado solo se calcula y se muestra si `$errores` está vacío. En una búsqueda por GET (ejercicios 2 y 3) no hay POST: se limpia `$_GET['q']` y se busca.

Plantilla para empezar cualquier ejercicio

Cópiala al empezar el examen y rellena los huecos. Es la estructura exacta de `ejercicio1.php` y `ejercicio4.php`:

```php
<?php
declare(strict_types=1);

/* ========== PASO 1 · DATOS ========== */
$xxx = [ /* datos del problema: monedas, películas... */ ];
$errores = [];     // un mensaje por campo que falla
$datos = [];       // un valor por campo correcto
$resultado = [];   // se rellena solo si no hay errores

/* ========== PASO 2 · FUNCIONES ========== */
function h(string $s): string
{
    return htmlspecialchars($s, ENT_QUOTES, 'UTF-8');
}

function validarXxx(string $texto): ?float      // devuelve el valor o null
{
    // ...
}

function gestionarErrores(array $entrada): array
{
    $errores = [];
    $datos = [];
    // por cada campo: validar -> $errores['campo'] o $datos['campo']
    return ['errores' => $errores, 'datos' => $datos];
}

/* ========== PASO 3 · FLUJO ========== */
$entrada = ['campo' => ''];                      // valores de la primera
visita (GET)

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // ETAPA 1 · LIMPIEZA
    $entrada = ['campo' => trim($_POST['campo'] ?? '')];

    // ETAPAS 2 y 3 · VALIDACIÓN Y ERRORES
    $validacion = gestionarErrores($entrada);
    $errores = $validacion['errores'];
    $datos = $validacion['datos'];

    // LÓGICA: solo si todo es correcto
    if (empty($errores)) {
        $resultado = [ /* cálculo con $datos */ ];
    }
}
```

```
?>
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>Título</title>
</head>
<body>
    <!-- ========== PASO 4 · SALIDA ========== -->
    <form method="post" action="<?= h($_SERVER['PHP_SELF']) ?>">
        <input type="text" name="campo" value="<?= h($entrada['campo']) ?>">
        <?php if (isset($errores['campo'])): ?>
            <span><?= h($errores['campo']) ?></span>
        <?php endif; ?>
        <button type="submit">Enviar</button>
    </form>

    <?php if (!empty($resultado)): ?>
        <!-- mostrar el resultado con h() y number_format -->
    <?php endif; ?>
</body>
</html>
```

Cuatro decisiones que debes tomar al leer el enunciado:

1. **GET o POST.** ¿Es una búsqueda? GET. ¿Se envían datos para calcular? POST.
2. **Qué arrays de datos** pide («crea un array donde…»): cada frase así es un array en el PASO 1.
3. **Qué se valida.** Si el enunciado no pide validar nada (buscador), las etapas 2 y 3 se omiten **y se dice en un comentario**, como hace el buscador refactorizado del zip.
4. **Qué se muestra** si no hay errores, y con qué formato (`number_format`, contador, imagen…).

Ejercicio 1 · Conversor de monedas, paso a paso

Enunciado resumido. Conversor con 3 monedas de origen y 3 de destino, en una sola página con `action` = `$_SERVER['PHP_SELF']`. Pide cuatro arrays (monedas y descripción, tipos de cambio, resultado y monedas, errores), validar una cantidad numérica mayor que cero con coma o punto, y usar `number_format` en la salida y `htmlspecialchars`.

Lectura del enunciado con las cuatro decisiones: POST (se envían datos para calcular) · 4 arrays en el PASO 1 · se validan cantidad, origen y destino · salida con `number_format`.

Paso 1 · Los cuatro arrays

Cada frase «crea un array donde…» del enunciado es un array:

```
<?php
declare(strict_types=1);

// ARRAY 1: moneda => descripción
$monedas = [
    'EUR' => 'Euro',
    'USD' => 'Dólar (USD)',
    'GBP' => 'Libra (GBP)',      // [dato de ejemplo inventado]
];

// ARRAY 2: moneda => tipo de cambio (unidades de esa moneda por 1 EUR)
$tiposCambio = [
    'EUR' => 1.00,
    'USD' => 1.10,               // sale de la captura: 100 EUR = 110 USD
    'GBP' => 0.85,               // [dato de ejemplo inventado]
];

$resultado = [];   // ARRAY 3: resultado y monedas (se rellena si todo va
bien)
$errores = [];     // ARRAY 4: errores
$datos = [];       // campos correctos
```

Por qué el cambio «por 1 EUR»: así cualquier pareja se convierte pasando por el euro, con una sola fórmula: `cantidad / cambio[origen] * cambio[destino]`. Ejemplo: 100 GBP → USD = 100 / 0,85 × 1,10 = 129,41.

Paso 2 · Las funciones

```
function h(string $s): string
{
    return htmlspecialchars($s, ENT_QUOTES, 'UTF-8');
}

// Etapa 2: número con coma o punto -> float o null
function validarNumero(string $texto): ?float
{
    $texto = str_replace(',', '.', $texto);
    $valor = filter_var($texto, FILTER_VALIDATE_FLOAT);
    return $valor === false ? null : $valor;
}

// Etapa 2: lista blanca. Solo vale un código que sea clave de $monedas
```

```
function validarMoneda(string $codigo, array $monedas): ?string
{
    return isset($monedas[$codigo]) ? $codigo : null;
}

// Etapas 2 y 3: un mensaje por cada fallo, sin parar en el primero
function gestionarErrores(array $entrada, array $monedas): array
{
    $errores = [];
    $datos = [];

    $cantidad = validarNumero($entrada['cantidad']);
    if ($cantidad === null) {
        $errores['cantidad'] = 'Introduce una cantidad numérica válida.';
    } elseif ($cantidad <= 0) {
        $errores['cantidad'] = 'La cantidad debe ser mayor que cero.';
    } else {
        $datos['cantidad'] = $cantidad;
    }

    $origen = validarMoneda($entrada['origen'], $monedas);
    if ($origen === null) {
        $errores['origen'] = 'Elige una moneda de origen válida.';
    } else {
        $datos['origen'] = $origen;
    }

    $destino = validarMoneda($entrada['destino'], $monedas);
    if ($destino === null) {
        $errores['destino'] = 'Elige una moneda de destino válida.';
    } else {
        $datos['destino'] = $destino;
    }

    return ['errores' => $errores, 'datos' => $datos];
}

// Lógica: conversión pasando por el euro
function convertir(float $cantidad, string $origen, string $destino, array
$tiposCambio): float
{
    return $cantidad / $tiposCambio[$origen] * $tiposCambio[$destino];
}
```

Claves de este paso:

- **Por qué validar el `select`** si el usuario solo puede elegir de la lista: cualquiera puede modificar el formulario o enviar `origen=XXX` a mano. Por eso el control lo hace PHP (pptx, diap. 3: «Sin required ni pattern: el control lo hace PHP»).
- `$monedas` y `$tiposCambio` se **pasan como parámetro** porque dentro de la función no se ven.

Paso 3 · El flujo: GET o POST

```
// Primera visita (GET): cantidad vacía y EUR -> USD, como en la captura
$entrada = ['cantidad' => '', 'origen' => 'EUR', 'destino' => 'USD'];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // ETAPA 1 · LIMPIEZA
    $entrada = [
        'cantidad' => trim($_POST['cantidad'] ?? ''),
        'origen' => trim($_POST['origen'] ?? ''),
        'destino' => trim($_POST['destino'] ?? ''),
    ];

    // ETAPAS 2 y 3
    $validacion = gestionarErrores($entrada, $monedas);
    $errores = $validacion['errores'];
    $datos = $validacion['datos'];

    // LÓGICA -> ARRAY 3
    if (empty($errores)) {
        $resultado = [
            'cantidad' => $datos['cantidad'],
            'origen' => $datos['origen'],
            'destino' => $datos['destino'],
            'convertido' => convertir($datos['cantidad'], $datos['origen'],
$datos['destino'], $tiposCambio),
            'detalle' => $monedas[$datos['origen']] . ' a ' .
$monedas[$datos['destino']],
        ];
    }
}
?>
```

Paso 4 · La salida (HTML)

```
<!DOCTYPE html>
<html lang="es">
<head><meta charset="utf-8"><title>Conversor de monedas</title></head>
<body>
    <h1>Conversor de monedas (única página)</h1>

    <!-- Errores arriba, como en la captura "Salida con errores" -->
    <?php if (!empty($errores)): ?>
        <ul>
            <?php foreach ($errores as $mensaje): ?>
                <li><?= h($mensaje) ?></li>
            <?php endforeach; ?>
        </ul>
    <?php endif; ?>

    <form method="post" action="<?= h($_SERVER['PHP_SELF']) ?>">
        <p>
            <label for="cantidad">Cantidad:</label>
            <input type="text" id="cantidad" name="cantidad" value="<?=
h($entrada['cantidad']) ?>">
        </p>
        <p>
            <label for="origen">Origen:</label>
            <select id="origen" name="origen">
                <?php foreach ($monedas as $codigo => $descripcion): ?>
                    <option value="<?= h($codigo) ?>" <?= $entrada['origen']
=== $codigo ? 'selected' : '' ?>>
                        <?= h($codigo . ' - ' . $descripcion) ?>
                    </option>
                <?php endforeach; ?>
            </select>
        </p>
        <!-- el select de destino es igual cambiando 'origen' por 'destino' -
->
        <p><button type="submit">Convertir</button></p>
    </form>

    <?php if (!empty($resultado)): ?>
        <p><strong>Resultado:</strong></p>
        <p>
            <?= number_format($resultado['cantidad'], 2, ',', '.') ?> <?=
h($resultado['origen']) ?>
            equivalen a
            <?= number_format($resultado['convertido'], 2, ',', '.') ?> <?=
```

```
h($resultado['destino']) ?>
        </p>
        <p>Detalle: <?= h($resultado['detalle']) ?></p>
    <?php endif; ?>
</body>
</html>
```

- El `type="text"` (y no `number`) es a propósito [no está en el material]: con `number` el navegador no dejaría escribir la coma y quien valida es PHP.
- Las opciones salen del array con `foreach`: añadir una moneda es añadir una línea en los dos arrays.

Pruebas que hice (servidor PHP 8.3 + curl)

Entrada	Salida
Primera visita	Formulario vacío con EUR y USD marcados
100, EUR → USD	100,00 EUR equivalen a 110,00 USD · Detalle: Euro a Dólar (USD)
`12,5` y `12.5`	Las dos: 12,50 EUR equivalen a 13,75 USD
100, GBP → USD	100,00 GBP equivalen a 129,41 USD
`abc`, vacío, `1,234.5`	Introduce una cantidad numérica válida.
`0` y `-5`	La cantidad debe ser mayor que cero.
origen `XXX` (manipulado)	Elige una moneda de origen válida.
`"><script>alert(1)</script>`	Se muestra escapado, no se ejecuta

Qué he usado y de dónde sale

Función o técnica	Documento
`$_SERVER['PHP_SELF']` en el `action`	Enunciado (`UT03-Ejercicios_9.pdf`); `Ejercicio2Original.php` (zip del buscador)
`$_SERVER['REQUEST_METHOD']`	UT03-Programación código embebido.pdf, pág. 22; pptx, diap. 9
`trim(... ?? '')`	pptx, diap. 4; UT03, pág. 59
`str_replace`	UT03, pág. 15

Función o técnica	Documento
`filter_var` + `FILTER_VALIDATE_FLOAT` + `=== false`	pptx, diap. 5; zip `EjemplosFilter_var`
`isset($array[$clave])` como lista blanca	UT03, pág. 21
Funciones con tipos y `?float` / `null`	UT03, págs. 8-12; `Validaciones.php` (zip `FormularioAutogestionado`)
`gestionarErrores()` que devuelve `['errores', 'datos']`	`funciones.php` (zip `06-Ejercicio8hoja8`)
`htmlspecialchars` y `h()`	UT03, págs. 60-61
`foreach`, `if/endif`, `foreach/endforeach`, `<?= ?>`	UT03, pág. 6; Chuleta de PHP, pág. 2; zips
`number_format`	[lo exige el enunciado, no aparece en los apuntes]

No cubierto por el material: `number_format` (lo exige el enunciado); la libra (GBP, 0,85) es un [dato de ejemplo inventado]; la conversión pasando por el euro es una decisión mía.

Ejercicio 2 · Buscador de películas, paso a paso

Enunciado resumido. Array asociativo con al menos 10 películas (título, año de estreno y sinopsis) y un formulario con una caja de texto que busca **solo por título**, en una única página.

Las cuatro decisiones: GET (es una búsqueda: la URL `?q=volver` se puede compartir) · un array anidado · **no se pide validar**, así que las etapas 2 y 3 se omiten y se dice en un comentario · se muestran las películas encontradas y cuántas son.

Paso 1 · El array anidado

La clave es el título (es por lo que se busca) y el valor, otro array con año y sinopsis. Son las 12 películas del zip del buscador; aquí van 3 para que veas la forma:

```php
<?php
declare(strict_types=1);

$peliculas = [
    'Volver' => [
        'anio' => 2006,
        'sinopsis' => 'Drama de Pedro Almodóvar sobre la familia, los
secretos y la supervivencia de varias mujeres en La Mancha.',
    ],
    'Tesis' => [
        'anio' => 1996,
        'sinopsis' => 'Suspense universitario sobre una estudiante que
investiga la morbosa fascinación por las imágenes violentas.',
    ],
    'Mar adentro' => [
        'anio' => 2004,
        'sinopsis' => 'Historia real de Ramón Sampedro, un hombre
tetrapléjico que luchó por su derecho a morir dignamente.',
    ],
    // ... hasta tener al menos 10
];
```

La clave es `'anio'` y no `'año'`: así está en la versión refactorizada del zip (la original, Ejercicio2Original.php, usaba 'año'). Cualquiera de las dos funciona; lo importante es usar la misma clave al crear el array y al leerlo.

Paso 2 · Las funciones

```php
function h(string $s): string
{
    return htmlspecialchars($s, ENT_QUOTES, 'UTF-8');
}

// Etapa 1: limpieza del parámetro GET "q" ($_GET es superglobal: no hace
falta pasarlo)
function limpiarBusqueda(): string
{
    return trim($_GET['q'] ?? '');
}

// Lógica: solo por título. Búsqueda vacía = sin resultados
function buscarPorTitulo(array $peliculas, string $q): array
{
    if ($q === '') {
        return [];
```

```
    }
    $encontradas = [];
    foreach ($peliculas as $titulo => $info) {
        if (mb_stripos($titulo, $q) !== false) {
            $encontradas[$titulo] = $info;   // se copia con la misma clave
        }
    }
    return $encontradas;
}
```

Cómo funciona la búsqueda: se recorre el array con `foreach (clave => valor)`; si el texto buscado aparece en el título (en cualquier posición y sin importar mayúsculas), la película se copia a `$encontradas`. Al terminar, `$encontradas` tiene la misma forma que `$peliculas`, así que la vista la recorre igual.

Paso 3 · El flujo

```
$q = limpiarBusqueda();                          // Etapa 1
// Etapas 2 y 3: validación y errores. No las pide el enunciado.
$resultados = buscarPorTitulo($peliculas, $q);  // Lógica
?>
```

No hay `if (REQUEST_METHOD === 'POST')`: con GET, la búsqueda viene en la URL y si no hay `q`, `??` deja `''` y no se muestra nada.

Paso 4 · La salida

```
<!DOCTYPE html>
<html lang="es">
<head><meta charset="utf-8"><title>Buscador (solo por título)</title></head>
<body>
    <h1>Buscador de películas por título</h1>

    <form method="get" action="<?= h($_SERVER['PHP_SELF']) ?>">
        <label for="q">Buscar por título:</label><br>
        <input id="q" name="q" type="text" value="<?= h($q) ?>">
        <button type="submit">Buscar</button>
        <?php if ($q !== ''): ?>
            <a href="<?= h($_SERVER['PHP_SELF']) ?>">Limpiar</a>
        <?php endif; ?>
    </form>

    <?php if ($q === ''): ?>
        <p>Introduce un título (o parte de él) para buscar.</p>
    <?php else: ?>
```

```
        <p><?= count($resultados) ?> resultado(s) para "<?= h($q) ?>".</p>
        <?php if (empty($resultados)): ?>
            <p>No se han encontrado películas con ese título.</p>
        <?php else: ?>
            <?php foreach ($resultados as $titulo => $info): ?>
                <article>
                    <div>
                        <?= h($titulo) ?>
                        <span>(<?= h((string) $info['anio']) ?>)</span>
                    </div>
                    <div><?= h($info['sinopsis']) ?></div>
                </article>
            <?php endforeach; ?>
        <?php endif; ?>
    <?php endif; ?>
</body>
</html>
```

- Hay tres situaciones en la salida: no se ha buscado nada, se ha buscado y no hay resultados, y hay resultados. Cada una tiene su rama del `if`.
- `(string) $info['anio']`: el año es `int` y `h()` pide `string` (modo estricto).

Pruebas que hice

Búsqueda	Salida
Sin buscar	Nota «Introduce un título…» y ningún resultado
`volver`	1 resultado(s) · **Volver** · (2006)
`espíritu`	Encuentra «El espíritu de la colmena» (tildes bien)
`LA`	5 resultados (no distingue mayúsculas)
`zzz`	0 resultado(s) y «No se han encontrado películas…»
`"><script>x</script>`	Escapado en la caja y en el mensaje

Qué he usado y de dónde sale

Función o técnica	Documento
Array anidado y `foreach ($a as $clave => $valor)`	UT03, págs. 6, 18 y 19

Función o técnica	Documento
`mb_stripos`	`Ejercicio2Original.php` y `src/buscador.php` (zip `06-EjercicioBuscadorPeliculas…`)
`limpiarBusqueda()` y `buscarPorTitulo()`	`src/entrada.php` y `src/buscador.php` (mismo zip)
`$_GET`, formulario por GET	UT03, págs. 57-58
`count()`	UT03, pág. 21
`empty()`	Chuleta de PHP, pág. 5
Casting `(string)`	UT02_Teoría, pág. 5

No cubierto por el material: nada.

Ejercicio 3 · Buscador con imágenes: qué se añade

Enunciado resumido. Copiar el ejercicio 2 y, para cada película encontrada, mostrar también **su imagen**, guardada en **otro array**. Mostrar el **número de películas encontradas** («1 resultado(s) para "tesis".»).

Es un ejercicio de «ampliar»: se copia el 2 entero y se tocan solo **tres sitios**. En el examen, si te piden ampliar un ejercicio anterior, haz lo mismo: localiza dónde cambian los datos y dónde cambia la salida, y no toques lo demás.

Qué cambia	Dónde	Qué se queda igual
Nuevo array `$imagenes`	PASO 1 · Datos	`$peliculas`
Un `<img>` en cada resultado	PASO 4 · Salida	`h()`, `limpiarBusqueda()`, `buscarPorTitulo()`
Contador en negrita y título «Buscador de películas (BÚSQUEDA POR TÍTULO)»	PASO 4 · Salida	El formulario

Paso 1 · El segundo array: misma clave que `$peliculas`

El truco es usar **el título como clave** en los dos arrays. Así, cuando el `foreach` de resultados tiene `$titulo`, la imagen es simplemente `$imagenes[$titulo]`.

```
// [dato de ejemplo inventado]: rutas inventadas; pon tus imágenes en una
carpeta img/
$imagenes = [
    'Volver' => 'img/volver.jpg',
    'Tesis' => 'img/tesis.jpg',
    'Mar adentro' => 'img/mar-adentro.jpg',
    // ... una por cada película de $peliculas, con EXACTAMENTE el mismo
título
];
```

Si un título se escribe distinto en los dos arrays («mar adentro» frente a «Mar adentro»), `$imagenes[$titulo]` no existe y PHP muestra un aviso de clave indefinida; además, como h() pide un string y recibe null, la página se corta con un error de tipo (strict_types) [no está en el material, comprobado ejecutándolo].

Paso 4 · La salida: el contador y la imagen

```
<?php if ($q === ''): ?>
    <p>Introduce un título (o parte de él) para buscar.</p>
<?php else: ?>
    <!-- NÚMERO DE PELÍCULAS: count() sobre los resultados -->
    <p><strong><?= count($resultados) ?></strong> resultado(s) para "<?=
h($q) ?>".</p>

    <?php if (empty($resultados)): ?>
        <p>No se han encontrado películas con ese título.</p>
    <?php else: ?>
        <?php foreach ($resultados as $titulo => $info): ?>
            <article>
                <!-- IMAGEN: sale del segundo array usando el título como
clave -->
                <img src="<?= h($imagenes[$titulo]) ?>" alt="Cartel de <?=
h($titulo) ?>" width="100" height="150">
                <div>
                    <?= h($titulo) ?>
                    <span>(<?= h((string) $info['anio']) ?>)</span>
                </div>
                <div><?= h($info['sinopsis']) ?></div>
            </article>
        <?php endforeach; ?>
    <?php endif; ?>
<?php endif; ?>
```

- La ruta también se escapa con `h()`: se escapa **todo** lo que se imprime, venga de donde venga.

- `alt` es el texto que se ve si la imagen no carga. [no está en el material: tus apuntes solo listan la etiqueta `<img>`, no sus atributos].
- En `ejercicio3.php` hay un poco de CSS (`display: flex`) para poner la imagen a la izquierda; no lo pide el enunciado.

Pruebas que hice

Búsqueda	Salida
`tesis`	**1** resultado(s) · imagen `img/tesis.jpg` · Tesis (1996)
`la`	**5** resultado(s) y 5 imágenes
`Volver`, `Mar adentro`, `Celda 211`	Cada una con su ruta correcta del array
`zzz`	**0** resultado(s) y el mensaje de «no encontradas»

Las imágenes se verán rotas hasta que pongas las tuyas en `img/` con esos nombres.

Qué he usado y de dónde sale

Función o técnica	Documento
Todo lo del ejercicio 2	Ver la tabla del ejercicio 2
Array asociativo título → ruta	UT03, págs. 17-19
`count()`	UT03, pág. 21
Etiqueta `<img>`	Chuleta de etiquetas HTML (documento del proyecto), sección 5; UT03, pág. 55 la menciona

No cubierto por el material: los atributos `src`, `alt`, `width` y `height` de `<img>` (necesarios para lo que pide el enunciado); las rutas de las imágenes son [dato de ejemplo inventado].

Ejercicio 4 · Sumar 10 números, paso a paso

Enunciado resumido. Formulario con 10 cajas con los números del 1 al 10, **creadas dinámicamente**. Un botón «Sumar» envía **un array** con esos números a la misma página y se calcula la suma.

Las cuatro decisiones: POST · datos: cuántas cajas y sus valores iniciales · se valida **cada** caja · se muestra la suma.

Dos ideas nuevas: «creadas dinámicamente» = las cajas las escribe **un bucle**, no tú a mano; «se enviará un array» = todas se llaman `numeros[]`.

Paso 1 · Datos

```
<?php
declare(strict_types=1);

$totalCajas = 10;   // cambiando este número cambia todo el formulario
$errores = [];      // clave = POSICIÓN de la caja (0..9)
$datos = [];
$suma = null;       // null hasta que se calcula
```

Paso 2 · Funciones

```
function h(string $s): string
{
    return htmlspecialchars($s, ENT_QUOTES, 'UTF-8');
}

// Etapa 2: entero -> int o null (FILTER_VALIDATE_INT, como el "array de
enteros" del zip)
function validarEntero(string $texto): ?int
{
    $valor = filter_var($texto, FILTER_VALIDATE_INT);
    return $valor === false ? null : $valor;
}

// Etapas 2 y 3: recorre las cajas y valida CADA una
function gestionarErrores(array $entrada): array
{
    $errores = [];
    $datos = [];
    foreach ($entrada as $posicion => $texto) {
        $numero = validarEntero($texto);
        if ($numero === null) {
            $errores[$posicion] = 'Debe ser un número entero.';
        } else {
            $datos[$posicion] = $numero;
        }
    }
    return ['errores' => $errores, 'datos' => $datos];
}

// Lógica: acumular con +=
```

```
function sumar(array $numeros): float
{
    $suma = 0;
    foreach ($numeros as $numero) {
        $suma += $numero;
    }
    return $suma;
}
```

La clave de `$errores` es la **posición** de la caja (0, 1, 2…), igual que en `$_POST['numeros']`. Eso permite pintar cada error justo al lado de su caja.

Paso 3 · Flujo

```
// GET: valores iniciales 1..10
$entrada = range(1, $totalCajas);

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // ETAPA 1 · LIMPIEZA
    $numeros = $_POST['numeros'] ?? [];
    if (!is_array($numeros)) {          // alguien envió numeros=texto
        $numeros = [];
    }

    // SIEMPRE 10 posiciones, falten o no datos
    $entrada = [];
    for ($i = 0; $i < $totalCajas; $i++) {
        $entrada[$i] = trim($numeros[$i] ?? '');
    }

    // ETAPAS 2 y 3
    $validacion = gestionarErrores($entrada);
    $errores = $validacion['errores'];
    $datos = $validacion['datos'];

    // LÓGICA
    if (empty($errores)) {
        $suma = sumar($datos);
    }
}
?>
```

Por qué el `for` de 10 vueltas: si alguien envía solo 3 números, las otras 7 posiciones quedan como `''` y dan error, en vez de sumar solo 3. Y por qué `is_array`: el zip `EjemplosFilter_var` lo comprueba siempre antes de recorrer un array que viene del

formulario.

Paso 4 · Salida: las cajas las crea el bucle

```
<!DOCTYPE html>
<html lang="es">
<head><meta charset="utf-8"><title>Sumar 10 números</title></head>
<body>
    <h1>Sumar <?= $totalCajas ?> números</h1>

    <form method="post" action="<?= h($_SERVER['PHP_SELF']) ?>">
        <?php foreach ($entrada as $posicion => $valor): ?>
            <p>
                <input type="text" name="numeros[]" value="<?= h((string)
$valor) ?>">
                <?php if (isset($errores[$posicion])): ?>
                    <span><?= h($errores[$posicion]) ?></span>
                <?php endif; ?>
            </p>
        <?php endforeach; ?>
        <p><button type="submit">Sumar</button></p>
    </form>

    <?php if ($suma !== null): ?>
        <p>La suma de los <?= $totalCajas ?> números es <strong><?=
h((string) $suma) ?></strong>.</p>
    <?php endif; ?>
</body>
</html>
```

- El `foreach` sobre `$entrada` escribe 10 `<input>`. En GET contienen 1..10 (de `range`) y tras un envío, lo que escribió el usuario.
- `(string) $valor`: en GET los valores de `range()` son `int`, y `h()` pide `string`.
- `$suma !== null` y no `if ($suma)`: una suma de 0 es válida y debe mostrarse.

Pruebas que hice

Envío	Salida
Primera visita	10 cajas con 1, 2, …, 10
Enviar sin tocar	La suma de los 10 números es **55**
`0` con espacios, `-3`, `1000`, ceros y `3`	Suma **1000**

Envío	Salida
Una caja vacía, `abc` y `1,5`	3 errores «Debe ser un número entero.» al lado de su caja; lo escrito se conserva
Solo 3 cajas enviadas	7 errores
`numeros=zzz` o POST vacío	10 errores, sin avisos de PHP

Qué he usado y de dónde sale

Función o técnica	Documento
`name="numeros[]"` y recibir un array	`formularioFilterVar.html` y `validacionesFilterVar.php` (zip `05-EjemplosFilter_var`)
`is_array`, `foreach` y `FILTER_VALIDATE_INT` por cada elemento	`validacionesFilterVar.php` (caso «ARRAY DE ENTEROS»); UT03, pág. 52
`for`	UT03, pág. 6
`range()`	UT03, págs. 20-21
`+=`	UT03, págs. 5-6; Chuleta de PHP, pág. 8
`isset`, `empty`, `h()`, `REQUEST_METHOD`, `trim(... ?? '')`	Igual que en el ejercicio 1

No cubierto por el material: nada. Decisión mía: validar enteros (como el ejemplo del zip). Si el profesor quiere decimales, cambia `validarEntero()` por la `validarNumero()` del ejercicio 1.

Errores típicos y checklist antes de entregar

Errores que restan puntos

Error	Qué pasa	Cómo se hace bien
Leer `$_POST['campo']` sin `??`	Aviso «Undefined array key» si el campo no llega	`trim($_POST['campo'] ?? '')` (UT03, pág. 59)
`if (!$valor)` tras `filter_var`	Rechaza el `0`, que es válido	`if ($valor === false)` (pptx, diap. 5)

Error	Qué pasa	Cómo se hace bien
`if (mb_stripos(...))`	No encuentra coincidencias al principio (posición 0)	`!== false`
Imprimir sin `h()`	XSS: el navegador ejecuta lo que escriba el usuario	`<?= h($valor) ?>` en todo, también en `PHP_SELF` (UT03, págs. 60-61)
Sticky value desde `$datos`	El campo con error aparece vacío	`value` desde `$entrada` (pptx, diap. 10)
Usar un array de fuera dentro de una función sin pasarlo	La variable no existe dentro (ámbito local)	Pasarlo como parámetro (UT02_Teoría, pág. 8; UT03, pág. 8)
Parar en el primer error	El usuario solo ve un fallo cada vez	Validar todos los campos y acumular en `$errores`
Mostrar el resultado aunque haya errores	Se calcula con datos que no existen	`if (empty($errores))` antes de calcular y mostrar
No validar el `select`	Se acepta un valor manipulado como `XXX`	Lista blanca con `isset($array[$codigo])` o `in_array`
`foreach` sobre `$_POST['numeros']` sin comprobar	Si llega un texto, error de PHP	`is_array()` antes (zip `EjemplosFilter_var`)
Olvidar `endif;` / `endforeach;`	Error de sintaxis: la página entera falla	Cerrar cada bloque al abrirlo
Pasar un `int` a `h()`	`TypeError` con `strict_types`	`h((string) $numero)` (UT03, pág. 12)

Dos trampas que encontré al probar [no están en el material, comprobadas ejecutándolo]

1. **Claves que parecen números se convierten en enteros.** En `['1' => '1ª evaluación', '2' => '2ª evaluación']`, PHP guarda las claves como `1` y `2` (int). Al recorrerlo con `foreach` y hacer `h($codigo)`, `strict_types` corta la página con un `TypeError`. Solución: claves de texto (`'primera'`, `'segunda'`) o `h((string) $codigo)`.

2. **Copiar código de la presentación.** La diap. 8 de la pptx tiene `$mensajes['email’]` con una comilla tipográfica (’) y la diap. 9 escribe `$datos[];` y usa `$datos` dentro de `v()` sin recibirlo. Si lo copias tal cual, no funciona: usa siempre comillas rectas (`'`), `$datos = [];` y pasa los arrays como parámetro.

Checklist antes de entregar

- [ ] Primera línea: `declare(strict_types=1);`
- [ ] Todos los arrays que pide el enunciado existen con un comentario que diga qué son
- [ ] `$errores = [];` y `$datos = [];` inicializados
- [ ] `h()` definida y usada en **todo** lo que se imprime (valores, errores, resultados, opciones, rutas, `PHP_SELF`)
- [ ] `action="<?= h($_SERVER['PHP_SELF']) ?>"` si el enunciado lo pide
- [ ] `if ($_SERVER['REQUEST_METHOD'] === 'POST')` para procesar solo al enviar (salvo búsquedas por GET)
- [ ] Cada `$_POST` / `$_GET` leído con `trim(... ?? '')`
- [ ] Cada campo validado con su función y su mensaje; comparaciones con `=== false` / `=== null`
- [ ] Los `select` validados contra su array (lista blanca)
- [ ] Los arrays del formulario comprobados con `is_array()` y validados elemento a elemento
- [ ] Los campos devuelven lo escrito (`value`) y el `select` marca la opción elegida (`selected`)
- [ ] El resultado solo se calcula y muestra si `empty($errores)`
- [ ] `number_format` en los números de salida si el enunciado lo pide
- [ ] Comentarios que señalan las cuatro etapas (las etapas omitidas, también comentadas)
- [ ] Lo has probado: primera visita, un caso correcto, un caso con cada error, un campo vacío y un `<script>`

Ejercicio tipo examen · Calculadora de nota media

Hazlo en un solo archivo `examen.php` , sin mirar las soluciones, en unos 60 minutos. Mezcla piezas de los cuatro ejercicios de la hoja 9. Lo he resuelto y probado yo antes para asegurarme de que se puede hacer **solo con lo que hay en tus apuntes**; la solución no está aquí.

Enunciado

Realiza una calculadora de la nota media de un alumno en **una sola página PHP**, con el `action` mediante `$_SERVER['PHP_SELF']` y el método POST.

- Crea un array donde almacenar las **asignaturas**: código y nombre. Usa estas cuatro: `DWES` → Desarrollo web en entorno servidor, `DWEC` → Desarrollo web en entorno cliente, `DIW` → Diseño de interfaces web, `DAW` → Despliegue de aplicaciones web.
- Crea otro array con las **evaluaciones** (código y texto): 1ª evaluación, 2ª evaluación y Evaluación final. Se eligen en un `select` generado desde el array.
- El formulario tendrá un campo **Alumno** (nombre) y **una caja de nota por asignatura**. Las cajas de nota deben **crearse dinámicamente** recorriendo el array de asignaturas, con el nombre de la asignatura como etiqueta, y se enviarán como **un array**.
- Valida:
  - El nombre: obligatorio, solo letras y espacios.
  - La evaluación: tiene que ser una de las del array.
  - Cada nota: numérica, entre **0 y 10**, admitiendo decimales con «,» o «.». Dos mensajes distintos: no numérica y fuera de rango.
- Crea un array donde almacenar los **errores**. Cada error debe aparecer **junto a su campo** y el formulario debe conservar lo escrito.
- Crea un array donde almacenar el **resultado**: nombre, evaluación, notas, media y número de asignaturas aprobadas.
- Si no hay errores, muestra:
  - Un título con el nombre del alumno y la evaluación.
  - Una tabla con cada asignatura, su nota y «Aprobado» (nota ≥ 5) o «Suspenso».
  - La nota media y «Asignaturas aprobadas: X de 4».
- Usa `number_format` con 2 decimales para las notas y la media, y `htmlspecialchars` para todo lo que muestres.

Pistas (solo si te atascas)

1. Empieza copiando el molde de las cuatro etapas y rellena primero el PASO 1 con los arrays.
2. Las cajas: `foreach ($asignaturas as $codigo => $nombre)` y dentro un `<input name="notas[]">`. Es el ejercicio 4 con un array asociativo en vez de `range()`.
3. En la limpieza llegan las notas como posiciones 0, 1, 2, 3. Para saber a qué asignatura pertenece cada una, recorre `$asignaturas` con un contador `$i` que empieza en 0 y sube con `$i++` en cada vuelta.

4. La nota se valida como la cantidad del ejercicio 1 (`str_replace` + `FILTER_VALIDATE_FLOAT`), pero el rango es 0 ⩽ nota ⩽ 10.
5. El nombre se valida con `validarNombre()` del zip `05-FormularioAutogestionadoConValidaciones`.
6. Media: acumula con `+=` en un `foreach` y divide entre `count()`. Aprobadas: un contador con `++` dentro de un `if`.
7. Ojo con las claves del array de evaluaciones (mira «Dos trampas que encontré al probar»).

Casos para autocorregirte

Son las salidas de mi solución. Si tu versión da lo mismo, vas bien.

#	Lo que envías	Lo que debe salir
1	Primera visita	Nombre vacío, 1ª evaluación seleccionada y 4 cajas vacías con el nombre de cada asignatura
2	Roberto Sainz · 1ª · `7` , `8,5` , `4.25` , `10`	7,00 / 8,50 / 4,25 / 10,00 · DIW suspenso · Nota media **7,44** · Asignaturas aprobadas: 3 de 4
3	ana · Final · `5` , `5,0` , `6.5` , `9,75`	Todas aprobadas (5 cuenta como aprobado) · Nota media **6,56**
4	Luis · 2ª · `0` , `10` , `0,0` , `10.00`	0 y 10 son válidos · Nota media **5,00** · 2 de 4
5	José Muñoz · 1ª · `5` , `6` , `7` , `8`	Acepta tilde y ñ en minúscula · Nota media **6,50**
6	`R2D2` · evaluación manipulada · vacía, `abc` , `11` , `-1`	6 errores: nombre, evaluación, dos «nota numérica» y dos «entre 0 y 10»; ningún resultado
7	Nombre vacío y notas correctas	Solo el error del nombre
8	ana · 2ª · `7` , `abc` , `11` , `4,5`	Errores en la 2ª y 3ª caja; las cajas conservan `7` , `abc` , `11` , `4,5` y sigue seleccionada la 2ª evaluación
9	Solo 2 notas enviadas	Las 2 cajas que faltan dan «nota numérica»

#	Lo que envías	Lo que debe salir
10	`"><script>x</script>` como nombre	Error del nombre y el texto se ve escapado en la caja, no se ejecuta

Dos cosas que verás y no son fallos tuyos [no están en el material, comprobadas ejecutándolo]:

- Con la expresión regular de `validarNombre()` de tus apuntes, «Ángel» (mayúscula con tilde) **se rechaza**, aunque «ángel» sí pasa.
- `1e1` como nota se acepta como 10 (notación científica, ver Fundamentos 7).

Ampliación (si te sobra tiempo)

Muestra debajo de la media qué asignatura tiene la nota más alta. Pista: recorre las notas guardando en dos variables la mejor nota vista hasta ahora y su código.

Cuando lo tengas, pásame tu `examen.php` y te lo corrijo con las mismas reglas: qué está bien, qué resta puntos y por qué.

Fuentes

Todo lo de esta guía sale de estos documentos del proyecto «Apuntes DWES», salvo lo listado en «No cubierto por el material».

Documento	Qué se ha sacado de él
`UT03-Ejercicios_9.pdf`	Enunciados de los ejercicios 1 a 4, `PHP_SELF`, `number_format`
`UT03-Programación código embebido.pdf`	if/elseif y ternario (págs. 2-3), while/for/foreach (5-6), funciones, `return`, parámetros y tipos (7-12), `strict_types` (12), `str_replace` (15), `mb_strlen` (17), arrays, `range`, `count`, `in_array`, `isset`, `unset` (17-21), `$_SERVER` y `REQUEST_METHOD` (22), tipos `is_array` (52), controles de formulario (53-55), GET/POST y `$_GET` / `$_POST` (56-58), `??` (59), `htmlspecialchars` y `h()` (60-61)

Documento	Qué se ha sacado de él
`UT02_Teoría.pdf`	Variables, tipos y casting (pág. 5), operadores y concatenación (6), ámbito de variables (8)
`Chuleta de PHP español con explicaciones.pdf`	`<?= ?>` (pág. 2), tipo anulable `?string` (4), `empty`, `isset` (5), `+=` y operadores (8-9), `trim`, `str_replace` (10)
`02-RA3-ValidacionFormularios.pptx`	Las cuatro etapas (diaps. 2 y 10), formulario de partida (3), `trim` (4), filtros y `=== false` (5-6), validaciones transversales (7), `$errores` / `$datos` (8), autogestionado (9)
Zip `05-EjemplosFilter_var`	`FILTER_VALIDATE_INT` con rango, `FILTER_VALIDATE_FLOAT`, `name="numeros[]"` + `is_array` + `foreach`
Zip `05-FormularioAutogestionadoConValidaciones`	`validarNombre` / `validarEdad` que devuelven valor o `null`, `FILTER_VALIDATE_REGEXP`, `if: endif` en el HTML
Zip `06-Ejercicio8hoja8_ValidadoYRefactorizado`	`gestionarErrores()` que devuelve `['errores', 'datos']`, lista blanca con `in_array`, `checked` en radios
Zip `06-EjercicioBuscadorPeliculasResueltoRefactorizado`	Array de películas, `limpiarBusqueda`, `buscarPorTitulo` con `mb_stripos`, `foreach: endforeach`, `PHP_SELF` escapado
`Chuleta de etiquetas HTML` (documento del proyecto)	Etiqueta `<img>`

No cubierto por el material

- `number_format`: [lo exige el enunciado, no aparece en los apuntes].
- Atributos `src`, `alt`, `width` y `height` de `<img>` (ejercicio 3).
- La libra (GBP) con cambio 0,85 y las rutas de las imágenes: [dato de ejemplo inventado].

- Comportamientos comprobados ejecutando el código y que los apuntes no explican: `FILTER_VALIDATE_FLOAT` no acepta la coma y sí `1e3`; `mb_stripos` devuelve 0 si la coincidencia está al principio; las claves numéricas en texto se convierten en enteros; una clave inexistente pasada a `h()` provoca un `TypeError`; la expresión regular de `validarNombre()` rechaza mayúsculas con tilde.
- El razonamiento de `type="text"` frente a `type="number"` en el ejercicio 1.